SafeStream-Redactor: Provably-Equivalent Constant-Memory Streaming Redaction of PII and Credentials

Anonymous Author(s)

Affiliation withheld for review
corresponding.author@example.org

Abstract. Redacting personally identifiable information (PII) and leaked credentials from very large text artefacts—logs, database dumps, and exports—forces an uncomfortable choice: load the whole document into memory (which does not scale) or process it in chunks (which risks cutting an entity in half at a chunk boundary). We present *SafeStream-Redactor*, a local-first redaction engine that resolves this tension with a *bounded-overlap* streaming architecture whose peak working memory is independent of input size, and we give a *formal equivalence guarantee*: under two explicitly stated locality assumptions—bounded span length and bounded context—the streamed output is identical to an idealised whole-document pass, and the multi-core output is byte-identical to the sequential pass or the engine transparently falls back to it through a seam cross-check. The guarantee is stated only for detectors that satisfy a bounded-window, chunk-independent contract, and the constants ($C_{\text{left}} = 48$, $C_{\text{right}} = 2$, $L_{\max} = 8192$) are read from the implementation rather than assumed. The detection layer is a calibrated ensemble of a deterministic regex-plus-validator tier, a Shannon-entropy tier for undocumented secrets, a contextual-scoring tier, and an optional named-entity tier, with a dependency-free logistic calibrator that turns hand-set confidences into probabilities. On a self-generated adversarial corpus dense with hard negatives, the deterministic tier holds precision 0.999 (45 benign false positives across 70,169 gold spans) with zero missed structured-PII entities; peak memory stays flat (2.75 MB) as input grows 4×, and a 1 GB file redacts under a 16 MB allocation cap. We are explicit about scope: these corpora are synthetic regression fixtures that measure format coverage and false-positive resistance, *not* real-world recall, and free-text recall is a precision/recall trade-off we characterise rather than claim to solve.

Keywords: PII redaction · secret detection · streaming algorithms · constant memory · formal equivalence · data privacy · named-entity recognition · confidence calibration.

1 Introduction and Contributions

Data-protection regulation (e.g. the EU General Data Protection Regulation [1]) and routine security hygiene both demand that sensitive values be removed from

text before it is shared, archived, or fed to downstream tooling. Two classes of sensitive value matter in practice: *PII* (emails, phone numbers, payment-card numbers, national identifiers, IP addresses, and free-form names and locations) and *credentials/secrets* (API keys, cloud access keys, service tokens, JWTs, private-key blocks, and bespoke high-entropy secrets). General-purpose PII engines such as Microsoft Presidio [2] and scrubadub target the former; dedicated secret scanners such as gitleaks [11] target the latter. Few tools cover both in a single pass, and—more importantly for this paper—none that we are aware of *states*, let alone *proves*, that its chunked or parallel execution produces the same output as processing the whole document at once.

That gap matters because the artefacts most in need of redaction are exactly the ones too large to hold in memory: multi-gigabyte application logs, incident captures, and data exports. A naive chunked redactor can silently miss an entity that straddles a chunk boundary, or emit different output depending on where the chunks happen to fall—a correctness defect that is invisible until it leaks.

Contributions. This paper makes the following contributions, each traceable to the SafeStream-Redactor implementation that is its sole source of truth:

1. **Constant-memory streaming redaction** of PII and credentials in one pass, with peak working memory bounded by $chunk_size + w + L_{\max}$ and independent of input size (Sections 4–5; Theorem 3).
2. **A bounded-overlap streaming architecture** that carries a rolling context window between chunks and retreats the emit boundary out of any straddling detection, so every entity up to the overlap length is seen whole at least once (Section 5).
3. **A formal equivalence guarantee** (Section 6) that the streamed output equals a whole-document reference pass under two explicitly stated assumptions—*bounded span length* (A1) and *bounded context locality* (A2)—whose constants are audited from the code. The theorem applies *only* to detectors satisfying a bounded-window, chunk-independent contract; we do *not* claim unconditional equivalence for arbitrary detectors.
4. **Safe multi-core execution**: a character-aligned range split with a seam cross-check that yields byte-identical output to the sequential pass, or transparently falls back to it on seam disagreement—making the parallel path equal to the sequential path *unconditionally* (Theorem 2).
5. **A calibrated detection ensemble** with a dependency-free logistic calibrator over detection-local features, and two named operating points—a precision-favouring STRICT policy and a recall-favouring HIGH_RECALL policy—exposed through a single confidence threshold (Section 7). We present the result as a precision/recall and benign-false-positive trade-off, not as a solution to free-text recall.

The novelty is not the engineering mechanisms (overlap windows, anchor prefilters, a seam protocol) individually, but the *guarantee* those mechanisms admit: a bounded-memory, parallel redactor whose output provably matches an idealised whole-document pass, with the one failure mode named and made safe.

2 Related Work

PII detection and de-identification. Microsoft Presidio [2] is a widely used open-source framework that combines pattern recognizers with a spaCy/transformer NLP pipeline for entity recognition. scrubadub offers a simpler rule- and model-based pipeline. In the clinical domain, Philter [3] performs rule-based de-identification of protected health information, and shared tasks such as the 2014 i2b2/UTHealth de-identification challenge [4] established the gold-standard evaluation methodology for longitudinal clinical narratives. Learned sequence labellers dominate free-text PII and named-entity recognition: the CoNLL-2003 shared task [5] remains a standard NER benchmark, transformer encoders such as BERT [6] set the modern baseline, and recent generalist models such as GLiNER [7] recognise arbitrary entity types from a prompt. Surveys of deep-learning NER [8] catalogue the precision/recall trade-offs inherent to free-text entity extraction. SafeStream is regex- and entropy-first by design, with named-entity recognition as an optional tier; it therefore trades lower free-text recall for zero required dependencies, local execution, and the equivalence guarantee below.

Secret and credential detection. Secret scanning is a distinct line of work. Meli et al. [9] characterised the prevalence of leaked secrets in public GitHub repositories, and curated corpora such as SecretBench [10] now support systematic evaluation. Tools such as gitleaks [11] and entropy-based heuristics—grounded in Shannon's measure of information [12]—detect both known-format keys and bespoke high-entropy secrets. General PII engines typically ship no credential recognizers at all; SafeStream covers both PII and secrets in the same streaming pass.

Streaming and parallel text processing. Constant-memory processing of unbounded input is the central concern of the data-stream model [13]. Practical scanning performance depends on the matching engine: Thompson's NFA construction [14] and the RE2 line of work [15] give linear-time regular-expression matching, though, as we report, lookaround-free and Unicode-boundary-consistent patterns are a prerequisite for adopting such engines without changing output. What is largely absent from prior redaction tools is an explicit statement that streaming or parallel execution is equivalent to a whole-document pass; SafeStream's bounded overlap and seam protocol are designed precisely to admit such a proof.

Transformation and pseudonymisation. Beyond deletion, redactors may mask or pseudonymise values. Keyed deterministic pseudonymisation (here built on HMAC [16]) preserves referential integrity across a corpus but, because equal inputs map to equal tokens, it leaks value equality and is not anonymisation in the sense of k-anonymity [17]; we treat it accordingly (Section 4).

3 Problem Definition and Design Goals

Problem. Given an input text T over a Unicode character alphabet, possibly far larger than available memory, produce a redacted text in which every detected sensitive span is rewritten by a redaction policy, while (i) using memory independent of $|T|$ and (ii) producing the *same* output a whole-document pass would produce.

Design goals.

G1 – Constant memory. Peak working memory must not grow with $|T|$, so a 100 GB file redacts in the same footprint class as a 1 KB one.
G2 – Boundary correctness. No entity may be missed or mis-scored merely because a chunk boundary falls inside it or inside the context it needs.
G3 – Equivalence under parallelism. Splitting one file across cores must yield byte-identical output to the sequential pass, or fail safely to it.
G4 – Dual coverage. PII *and* credentials/secrets must be detected in a single pass.
G5 – Local-first. Processing must run fully offline with no required third-party runtime dependency in the core, so sensitive text never has to leave the machine.
G6 – Honest, calibrated confidence. Detection confidences should be interpretable as probabilities and expose a principled precision/recall knob, rather than an arbitrary threshold.

Non-goals. We do not target the widest possible catalogue of built-in entity types, managed cloud scale, multilingual coverage, or enterprise data-governance workflows; these are better served by cloud DLP and governance platforms. We also do not claim to solve free-text recall: names, street addresses, and formatless identifiers have no distinctive shape, and any detector faces an irreducible precision/recall trade-off on them.

4 SafeStream-Redactor Architecture

Figure 1 shows the layered architecture. A coordination layer (`Redactor`) drives the streaming engine, which feeds text windows to a cascade of detection tiers whose outputs are scored, filtered, and overlap-resolved before a redaction policy rewrites the text.

Detection tiers (G4). Tier 1 is a set of regular expressions each paired with a validator, so a syntactic match alone is insufficient: payment-card candidates must pass the Luhn checksum [18], national-identifier candidates must satisfy area/-group rules, and IPv4/IPv6 candidates must parse as addresses. Tier 1 covers structured PII (email, phone, payment card, SSN, IPv4/IPv6) and a broad set of credential formats (AWS access keys and secret keys, GitHub, Slack, Stripe,

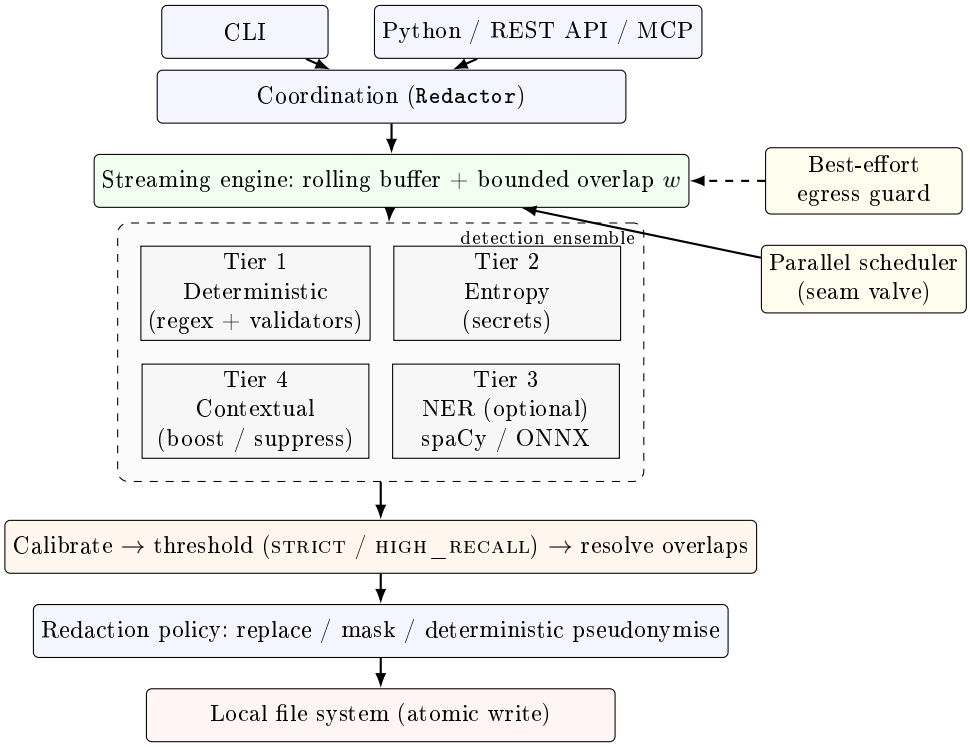


Fig. 1. SafeStream-Redactor architecture. The streaming engine presents bounded windows to a four-tier detection ensemble; detections are calibrated, thresholded under the selected policy, overlap-resolved, and rewritten. The egress guard is best-effort defence-in-depth, not a hard boundary; the parallel scheduler's seam valve guarantees a safe fallback.

Google, SendGrid, Twilio, npm, and OpenAI/Anthropic tokens, JWTs, private-key PEM blocks, and URL-embedded credentials). Tier 2 scores each candidate token's Shannon entropy [12] to catch bespoke, undocumented secrets with no published format, using per-alphabet entropy floors (base64 ≥ 4.0, hex ≥ 3.0 bits/char) and a minimum length to hold precision high. Tier 3 is an optional named-entity tier (spaCy, or a local ONNX token classifier) for PERSON/ORG/LOC. Tier 4 reads a bounded left window around each candidate and adjusts confidence—boosting values next to cue words ("ssn:", "card", "token") and suppressing values next to placeholder markers ("example", "test", "sample")—and promotes a few trigger-gated spans (e.g. a bare nine-digit number next to "ssn").

Redaction policy. A detected span is rewritten by one of three modes: *replace* with a fixed string, *mask* keeping the last k characters, or *deterministic pseudonymisation*. The last derives a keyed HMAC-SHA256 [16] token so that equal inputs map to equal tokens (referential integrity across a corpus).

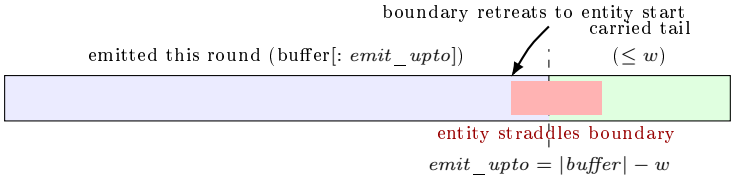


Fig. 2. Bounded-overlap emission. Only text at least w from the buffer end is emitted; an entity straddling the emit boundary pulls the boundary back to its start and is carried whole into the next window. Any span of length $\leq w$ is therefore seen intact at least once, regardless of where chunks fall.

We describe it as *deterministic keyed pseudonymisation*, not as reversible or anonymising: it deliberately leaks value equality, and an adversary who holds the key can brute-force a low-cardinality domain; it must be paired with a strong, secret key and is not k-anonymity [17].

Local-first execution (G5). The core library has no third-party runtime dependency; the NER and ONNX tiers are optional extras. During command-line redaction the tool installs a *best-effort* in-process network guard that raises on non-loopback socket use. We are explicit that this is defence-in-depth, *not* a hard security boundary: native extensions issuing raw syscalls, spawned subprocesses, or other processes on the machine are outside its reach. An opt-in, Linux-only hard mode (seccomp-bpf or a network namespace) is available for a kernel-enforced boundary, and whole-container isolation remains the strongest option; these are deployment features outside this paper's core claims.

5 Streaming Algorithm

The engine (Algorithm 1) consumes T as an *arbitrary* sequence of chunks and maintains a rolling buffer holding the current chunk plus a tail carried from the previous iteration. Detection runs on the whole buffer, but only the prefix lying at least w characters (the *overlap*) from the buffer's end is emitted; the remainder is carried forward. If a detection would straddle the planned emit boundary, the boundary retreats to that detection's start, so a match is never cut in half (Figure 2). Already-emitted text is retained, up to w characters, as read-only *left context*, so the contextual tier scores a span exactly as it would in whole-document mode.

Anchor prefilter. Each Tier-1 pattern declares literal *anchors* every match must contain (`AKIA`, `eyJ`, `@`, `://`, a digit, ...). A cheap substring scan per window skips any regex whose anchor is absent, so secret-sparse text runs markedly faster (about 25× on such text, measured) with *provably identical* output—a differential test compares prefiltered and unfiltered detection over fuzzed input. Prefiltering is an optimisation only; it never changes the detection set.

Algorithm 1 Bounded-overlap streaming (sequential engine S_w).

Require: chunk iterator; detector detect($\cdot$); overlap w
1: *prefix* $\leftarrow$ “”; *buffer* $\leftarrow$ “”
2: **for** each *chunk* **do**
3: *buffer* $\leftarrow$ *buffer* $\|$ *chunk*
4: **if** $|buffer| \leq w$ **then continue**
5: **end if**
6: *emit_upto* $\leftarrow |buffer| - w$
7: $D \leftarrow$ detect(*prefix* $\|$ *buffer*), offsets shifted by $-|prefix|$; drop spans with start < 0
8: **retreat: while** some $d \in D$ has $d.start < emit_upto < d.end$: *emit_upto* $\leftarrow$ min $d.start$
9: **if** *emit_upto* ≤ 0 **then continue**
10: **end if**
11: emit (*buffer*[:*emit_upto*], $\{d \in D : d.end \leq emit_upto\}$)
12: *prefix* $\leftarrow$ (*prefix* $\|$ *buffer*[:*emit_upto*])[$-w$:]
13: *buffer* $\leftarrow$ *buffer*[*emit_upto*:]
14: **end for**
15: emit the final (*buffer*, detect(*prefix* $\|$ *buffer*)) if non-empty

Parallel execution (G3). To use multiple cores on a single file, the engine splits T into k character-aligned byte ranges (respecting `\r\n` pairs and self-synchronising encodings). Each worker redacts its range with S_w extended by w characters of *scanned pad* and a further w of *context-only prefix* on each interior side, so every seam-crossing span is seen whole and with real left context. A span crossing a seam is owned by the *left* worker; both neighbours independently recompute the seam from the same bytes. An orchestrator compares each neighbour pair's seam reports and, on any disagreement, discards the partial outputs and redacts the whole file sequentially. This “seam valve” is what makes the parallel path safe: disagreement is only possible when a span longer than w sits exactly on a seam, and the valve converts that into a transparent fallback rather than silent character loss or duplication.

6 Formal Model and Equivalence Guarantees

We model the whole-document pipeline as a function and prove the streaming and parallel engines equal it under two locality assumptions whose constants are audited from the implementation. The guarantees hold *only* for pipelines that satisfy this bounded-window, chunk-independent contract.

6.1 Objects

A *detection* is a tuple (i, j, τ, c) with $0 \leq i < j \leq |T|$, type τ, and confidence $c \in [0, 1]$. A *detector* D : `str` $\rightarrow$ list of detections returns spans as offsets into the string it is given. The *reference pipeline* Π : `str` $\rightarrow$ set of detections runs every

Table 1. Locality constants audited from the implementation (characters). These define the detector contract under which Theorems 1–3 hold; they are read from the code, not assumed.

Constant	Value	Source in implementation
C_{left} (left context)	48	contextual scorer window
C_{right} (right context)	2	widest lookahead `(?!\.\d)` (phone, IPv4)
$L_{\max}$ (max span)	8192	private-key bound; entropy candidate cap
w (default overlap)	8192	engine `DEFAULT_OVERLAP`
chunk_size	65536	default read size (64 KiB)

configured tier on the whole T, applies contextual scoring over a bounded left window, drops detections below a threshold θ (`min_confidence`), and resolves overlaps into a sorted, pairwise-disjoint set. Redaction is a pure rewrite of T given $\Pi(T)$, so equality of detection sets implies byte-identical output; we therefore state the theorems over detection sets. The streaming engine S_w (Algorithm 1) and the parallel engine $P_{w,k}$ (k workers) are as defined in Section 5; $S_w(T)$ and $P_{w,k}(T)$ denote their emitted detections with offsets mapped back to absolute positions.

6.2 Assumptions and audited constants

A1 (bounded span length). Every detection in $\Pi(T)$ has length $j - i \leq L_{\max}$.

A2 (bounded context locality). For every detection $s = (i, j, \cdot, \cdot)$, whether $s \in \Pi(T)$ and its final confidence depend only on the substring $T[\, i - C_{\text{left}} : j + C_{\text{right}} \,]$.

A2 says the pipeline never needs more than C_{left} characters left or C_{right} right of a span to decide it; A1 says no span exceeds $L_{\max}$. Tracing each built-in tier's decision function fixes the constants (Table 1). The contextual scorer reads a 48-character left window; the widest right-context assertion is a two-character negative lookahead; every validator reads only the matched text. Two historically long patterns (private-key PEM bodies and high-entropy runs) are explicitly bounded to the default overlap: the private-key span is capped at exactly 8192 characters by bounding its key-type fields and body, and the entropy candidate is capped at 8192 with longer runs emitted as consecutive capped spans (covering the whole run with no gap). Hence $\max(C_{\text{left}}, C_{\text{right}}) = 48 \leq L_{\max} = 8192 = w =$ `DEFAULT_OVERLAP`, so the premise $w \geq \max(L_{\max}, C_{\text{left}}, C_{\text{right}})$ holds *unconditionally at the default configuration* for the built-in tiers.

6.3 Theorems

Theorem 1 (Streaming equivalence). *For any pipeline Π satisfying A1 and A2, if $w \geq \max(L_{\max}, C_{\text{left}}, C_{\text{right}})$ then $S_w(T) = \Pi(T)$ for every T and every chunking of T.*

Proof (Proof sketch). (a) Completeness. The engine emits only text at least w from the buffer end and retreats the emit boundary to the start of any straddling detection; the retreated tail, of length at most $L_{\max} \leq w$, is carried into the next window. Hence any span cut by a chunk boundary is reassembled and presented whole to detect in the next window. *(b) Soundness.* Each window is detect(*prefix* $\|$ *buffer*) where *prefix* is the last $w \geq C_{\text{left}}$ emitted characters, so every span's left context is real text. A span is emitted only when it ends at least $w \geq C_{\text{right}}$ characters before the buffer's live end, so its right context is real text too. By A2 the window verdict and confidence equal the whole-document values; detections falling inside *prefix* are dropped. *(c) Exactly once.* A span is emitted in the unique segment whose *emit_upto* first exceeds its end, and the carried region is disjoint from the emitted region. Together (a)–(c) give $S_w(T) = \Pi(T)$; byte-identity of output follows because the rewrite is a pure function of $(T, \Pi(T))$. □

Theorem 2 (Parallel byte-identity, with safe fallback). *Under A1 with $w \geq L_{\max}$, $P_{w,k}(T) = S_w(T)$ byte-for-byte. When A1 is violated at a seam, the seam valve detects the disagreement and $P_{w,k}$ falls back to S_w. Thus $P_{w,k}(T) = S_w(T)$* unconditionally*, and the parallel path never silently diverges from the sequential one.*

Proof (Proof sketch). Character-aligned ranges decoded with the same incremental decoder reproduce the sequential substring exactly. Each worker prepends w characters of scanned pad and w of context-only prefix, so with $w \geq L_{\max}$ (A1) and $w \geq C_{\text{left}}$ (A2) every seam-crossing span is seen whole with real left context—the Theorem 1 conditions applied per range. A seam-crossing span is owned by the left worker; both neighbours recompute the seam from identical bytes. The orchestrator compares seam reports and falls back to S_w on any mismatch. A mismatch is possible only when a span longer than w sits on a seam, i.e. exactly when A1 fails; so the valve turns every A1 violation into a transparent fallback. □

Theorem 3 (Constant-memory bound). *Peak working memory of S_w is at most chunk_size$+w+L_{\max}$, independent of $|T|$; $P_{w,k}$ uses at most k times that.*

Proof (Proof sketch). After each emit the engine retains only the carried region (at most w plus one straddling span of length $\leq L_{\max}$); the next iteration adds at most one *chunk_size* chunk before emitting again. The buffer therefore never exceeds *chunk_size* $+ w + L_{\max}$ regardless of $|T|$. Each worker runs one S_w plus a $2w$ holdback, so total memory scales with k, never with $|T|$. □

6.4 Where the guarantee stops

The assumptions fail in exactly these cases, all detectable or documented. (i) A span longer than w: no built-in tier produces one at the default overlap, but a user may lower w, add a custom detector with an unbounded span, or supply a single high-entropy run longer than w; in sequential mode such a span may be split, and in parallel mode the seam valve falls back. (ii) A detector needing unbounded

right context: A2 fixes $C_{\text{right}} = 2$, so any future tier that conditions on a whole document—most notably a large-language-model tier—breaks A2, and *neither theorem applies to it.* This is the principled reason the bounded, offline tiers are the ones that carry the equivalence guarantee. We note that the optional NER and ONNX tiers declare a bounded (256-character) context window but are statistical models whose effective context may exceed it near a chunk boundary; their accuracy there is a documented limitation the overlap mitigates but does not eliminate, and we do not extend the byte-exact equivalence claim to them. (iii) Non-splittable encodings fall back to the sequential path by construction, to which Theorem 1 still applies.

7 Semantic Detection Ensemble and Policy Calibration

Calibrated confidences (G6). The built-in tier confidences are hand-set constants that order detections sensibly but are not probabilities. To remove the "you just picked 0.5" objection, a dependency-free logistic *calibrator* maps each detection to a probability of being a true positive from detection-*local* features only: the raw confidence, a one-hot of the producing tier, whether context boosted or suppressed it, and whether its type is structured (checksum/grammar-backed) or free-form. Crucially every feature is a function of the detection alone, so applying the calibrator changes the reference pipeline Π but *preserves* the equivalence of $S_w/P_{w,k}$ to Π (Theorems 1–2). The accompanying harness trains the calibrator on a labelled corpus and reports Brier score and expected calibration error before versus after, plus a per-tier ablation (the marginal micro-F1 of the entropy and NER tiers). The calibrator is the mechanism; the labelled corpus and the resulting calibration numbers are supplied by the user under their own data licences and are *not* bundled in the repository, so we report the method rather than a specific trained model's scores.

Two operating points: strict and high_recall. The ensemble exposes a single threshold θ (`min_confidence`) that, together with the contextual tier, defines two named policies:

STRICT (default, $\theta = 0.5$). Precision-favouring. The contextual tier *subtracts* confidence from values adjacent to placeholder markers ("example", "test", "sample", ...), so documentation snippets and fixture data are left untouched. The side effect is that a file *saturated* with marker words under-detects—working as designed.

HIGH_RECALL ($\theta \approx 0.35$, NER enabled). Recall-favouring. Lowering the threshold recovers most values the placeholder suppression demotes, at some precision cost, and enabling the NER tier adds free-text names; intended for data known to be noisy rather than for trusted prose.

These are operating points on one precision/recall curve, selected per run, not separate detectors.

Named-entity tiers as an integration interface. Free-text PERSON/ORG/LOC recognition is delegated to an optional spaCy tier or a local ONNX token-classification tier. The ONNX tier is a *model-integration interface*: it provides a pure, unit-tested BIO/IOB2 decoder and pluggable tokeniser/backend seams, and runs any supplied on-device model under the offline guard with no data egress. The repository contains no benchmark of a concrete ONNX (or GLiNER-class) model, so we present ONNX strictly as an integration path and report *no* accuracy numbers for it; doing otherwise would misrepresent a mock as a measured result.

The trade-off, stated honestly. Perfect recall and perfect precision together are unattainable for free-text PII by any tool: a bank-account number is indistinguishable from any other run of digits, and "April" is both a name and a month. SafeStream is tuned to favour precision on structured PII and credentials, with names and format-less identifiers handled by the opt-in levers above. We therefore report results as a precision/recall and benign-false-positive trade-off (Section 8), and explicitly do *not* claim that semantic/free-text recall is solved.

8 Experimental Evaluation

Scope and honesty of the corpora. The corpora used for the accuracy numbers below are *synthetic and self-generated*: gold positives are emitted in the canonical formats the detector targets. A near-perfect score on them therefore measures *format coverage and false-positive resistance*, not real-world recall. They are a regression fixture, not an accuracy claim. The repository ships an evaluation harness (`evaluate.py`) with bootstrap confidence intervals, a paired significance test, CoNLL-2003/Presidio-dataset loaders, and baseline adapters (regex, Presidio with NER, scrubadub, gitleaks) for running on independent labelled corpora (Presidio research dataset, CoNLL-2003, the n2c2 clinical de-identification set [5,4]); those corpora carry their own licences and are supplied by the user, so the real-world accuracy study is enabled but not bundled. All numbers reported here are SafeStream's own measured values from the repository's recorded results; competitor rows are clearly labelled as to provenance.

Environment. Recorded results were produced on CPython 3.11.15, Linux x86-64, an Intel Xeon at 2.10 GHz (4 cores, a shared/throttled vCPU), 16 GB RAM, package version 0.2.0. Throughput is environment-specific on this throttled vCPU and should be read as *relative*, not a hardware ceiling.

8.1 Format coverage and false-positive resistance

Table 2 reports the deterministic tier on two self-generated corpora. Matching is type-aware with at least 50% span overlap; the types are email, phone, credit card, SSN, and IPv4—the set the deterministic tier and a pattern baseline both

Table 2. Measured accuracy and throughput on self-generated corpora (type-aware matching, $\geq 50\%$ overlap; types: email, phone, credit_card, ssn, ipv4). These measure format coverage and false-positive resistance, *not* real-world recall.

Corpus	Size	Gold	P	R*	F1	FP	FN	MB/s
Clean / canonical	10.5 MB	60,253	1.000	1.000	1.000	0	0	3.50
Adversarial (hard neg.)	5.2 MB	70,169	0.999	1.000	1.000	45	0	2.06

* Recall is against self-generated positives and is *not* a real-world recall figure; the non-circular result is the precision under distractors.

Table 3. Measured constant-memory scaling (peak `tracemalloc` allocations; $w = 8\,\text{KiB}$, $chunk_size = 64\,\text{KiB}$, single worker).

Input size	Peak memory	Ratio to smallest
8 MB	2.75 MB	1.00×
32 MB	2.75 MB	1.00×
1 GB (repository test)	< 16 MB (asserted)	—

cover. The clean-corpus F1 = 1.000 is a regression fixture and is close to circular. The *informative* number is the adversarial precision: on 5.2 MB dense with hard negatives (Luhn-failing 16-digit numbers, 5-octet version strings, invalid-area SSNs, ISO timestamps, UUIDs, git hashes), the validators reject almost all distractors, leaving only 45 benign false positives across 70,169 gold spans (P = 0.999). This is the benign-false-positive side of the trade-off in Section 7, measured.

8.2 Constant memory

Table 3 reports peak Python-heap allocations (via `tracemalloc`) as input grows. A 4× increase in input produces *zero* increase in peak allocation—the empirical signature of Theorem 3. A repository test corroborates the property three orders of magnitude larger: a ∼1 GB generated file redacts with peak allocations under a 16 MB cap.

8.3 Parallel byte-identity and scanning speed

Parallel byte-identity (Theorem 2) is enforced continuously: the test suite and the speedup harness assert that splitting a file across workers produces output byte-identical to the sequential pass at every worker count, or falls back via the seam valve. A prior run recorded about 3.7× speedup on 4 cores. The anchor prefilter yields about 25× on secret-sparse text with provably identical output. We are candid that raw single-core throughput (2–3.5 MB/s on this throttled vCPU) is a known limitation, not a strength: compiled scanners are orders of magnitude faster. A drop-in re2/Vectorscan backend was prototyped and measured but *not*

Table 4. Abridged capability comparison (documented; SafeStream measured where noted). • documented yes, ○ no/limited, ▲ optional/add-on, ? unknown. Full matrix against ten systems is in the repository.

Capability	SafeStream	Presidio	Philter	gitleaks-class	Cloud DLP
Runs fully offline / local	•	•	•	•	○
PII detection	•	•	•	▲	•
Credential / secret detection	•	○	○	•	▲
High-entropy secret detection	•	○	○	•	○
ML / NER detection	▲	•	•	○	•
Streaming / large-file input	•	○	?	•	•
Bounded / constant memory	• (meas.)	○	?	?	N/A
No data leaves the machine	•	•	•	•	○

adopted, because re2's ASCII-only word boundaries change detection output on Unicode text and it cannot compile the lookaround the throughput-dominating PII patterns use; the value here is constant memory, dual PII+secret coverage, and the equivalence guarantee, not speed.

8.4 Capability comparison with existing systems

Table 4 summarises a *documentation-based* capability comparison (abridged) against widely used systems. Only SafeStream's cells are *measured* in the repository; every competitor cell reflects *documented* capability from public sources and is labelled as such. A blank or "?" means *not documented*, not "no". Separately, a prior head-to-head run—recorded in the project when Presidio was installed—found SafeStream at precision 0.999 versus Presidio's pattern recognizers at 0.649 (recall 0.843) on the adversarial corpus, and 9/9 versus 0/9 on a credential corpus; we report these as a *prior measured* comparison and a *capability* difference (Presidio ships no credential recognizers), not as a controlled side-by-side produced in the same environment as Table 2. Presidio's spaCy/-transformer NER tier is a separate, model-dependent path that out-recalls a regex core on free-text names.

9 Discussion and Threats to Validity

Construct validity. The headline accuracy corpora are synthetic and self-generated; their near-perfect recall is by construction and is *not* evidence of real-world recall. We mitigate this by (i) reporting precision under adversarial distractors as the informative number, (ii) framing the clean corpus explicitly as a regression fixture, and (iii) shipping a harness that runs the same scoring on independent labelled corpora with confidence intervals and significance tests. The honest state of a regex-first detector is high recall on structured PII and

credentials and substantially lower recall on free-form names/addresses without the NER tier.

Internal validity. The equivalence theorems depend on assumptions A1/A2 and apply only to detectors satisfying the bounded-window, chunk-independent contract. We audited the constants from the code rather than assuming them, and the built-in tiers satisfy the contract at the default overlap; a custom detector or a lowered overlap can violate A1, which in parallel mode is caught by the seam valve and in sequential mode is surfaced by a runtime warning. Statistical NER tiers are outside the byte-exact claim by design.

External validity. Throughput was measured on a single throttled vCPU and is relative; the capability comparison against other systems is documentation-based except for the prior Presidio run, and cloud/commercial systems were not benchmarked (they require credentials, cost approval, and terms permitting publication). We therefore label measured, prior-measured, and documented claims distinctly throughout.

Security caveats. The in-process network guard is best-effort defence-in-depth, not a hard boundary; a kernel-enforced mode and container isolation are the stronger options. Deterministic keyed pseudonymisation preserves referential integrity but leaks value equality and is invertible by a key holder over low-cardinality domains; it is not anonymisation. These are stated so that the guarantees are not over-read.

10 Conclusion

SafeStream-Redactor shows that a redactor can be simultaneously *constant-memory*, *parallel*, and *provably output-equivalent* to a whole-document pass—under assumptions that are stated precisely, audited from the implementation, and bounded by a named, safely-handled failure mode. It covers PII and credentials in a single local-first pass, and exposes a calibrated precision/recall knob with STRICT and HIGH_RECALL operating points. We deliberately do not claim to solve free-text recall, and we separate measured from documented results throughout. The contribution is the guarantee the engineering admits: a bounded-memory parallel redactor whose output matches an idealised whole-document pass, with the one place the guarantee stops made explicit and safe. Future work is the independent-corpus accuracy study the bundled harness already supports, and principled integration of statistical and model-based tiers that, by design, fall outside the byte-exact equivalence claim.

References

1. European Parliament and Council of the European Union: Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union L119, 1–88 (2016)

2. Microsoft: Presidio — Data Protection and De-identification SDK. https://github.com/microsoft/presidio (last accessed 2026)
3. Norgeot, B., Muenzen, K., Peterson, T.A., Fan, X., Glicksberg, B.S., Schenk, G., Rutenberg, E., Oskotsky, B., Sirota, M., Yazdany, J., Schmajuk, G., Ludwig, D., Goldstein, T., Butte, A.J.: Protected Health Information Filter (Philter): Accurately and Securely De-identifying Free-text Clinical Notes. npj Digital Medicine 3, 57 (2020)
4. Stubbs, A., Kotfila, C., Uzuner, Ö.: Automated Systems for the De-identification of Longitudinal Clinical Narratives: Overview of 2014 i2b2/UTHealth Shared Task Track 1. Journal of Biomedical Informatics 58, S11–S19 (2015)
5. Tjong Kim Sang, E.F., De Meulder, F.: Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In: Proc. CoNLL-2003, pp. 142–147 (2003)
6. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: Proc. NAACL-HLT, pp. 4171–4186 (2019)
7. Zaratiana, U., Tomeh, N., Holat, P., Charnois, T.: GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer. In: Proc. NAACL-HLT, pp. 5364–5376 (2024)
8. Li, J., Sun, A., Han, J., Li, C.: A Survey on Deep Learning for Named Entity Recognition. IEEE Transactions on Knowledge and Data Engineering 34(1), 50–70 (2022)
9. Meli, M., McNiece, M.R., Reaves, B.: How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories. In: Proc. Network and Distributed System Security Symposium (NDSS) (2019)
10. Basak, S.K., Neil, L., Reaves, B., Williams, L.: SecretBench: A Dataset of Software Secrets. In: Proc. IEEE/ACM 20th International Conference on Mining Software Repositories (MSR), pp. 347–351 (2023)
11. Rice, Z.: gitleaks — Find secrets and detect hardcoded credentials. https://github.com/gitleaks/gitleaks (last accessed 2026)
12. Shannon, C.E.: A Mathematical Theory of Communication. The Bell System Technical Journal 27(3), 379–423 (1948)
13. Muthukrishnan, S.: Data Streams: Algorithms and Applications. Foundations and Trends in Theoretical Computer Science 1(2), 117–236 (2005)
14. Thompson, K.: Programming Techniques: Regular Expression Search Algorithm. Communications of the ACM 11(6), 419–422 (1968)
15. Cox, R.: Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html (2007, last accessed 2026)
16. Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-Hashing for Message Authentication. RFC 2104, IETF (1997)
17. Sweeney, L.: k-anonymity: A Model for Protecting Privacy. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems 10(5), 557–570 (2002)
18. Luhn, H.P.: Computer for Verifying Numbers. U.S. Patent 2,950,048 (1960)